

Towards Quantum Machine Learning for Quartet Topology Inference: A Quantum Kernel Approach

Ho Thu Giang¹, Sy-Vinh Le¹, Le Thi Huong¹, Le Kim Thu², Thang N. Dinh³, and Nguyen-Viet-Dung Nghiem¹

¹Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Vietnam

²Faculty of Mathematics and Informatics, Hanoi University of Science and Technology

³Department of Computer Science, Virginia Commonwealth University, Richmond, VA, USA

Abstract—Quartet phylogenetic inference is a fundamental task in phylogenetic analysis to reconstruct the evolutionary relationships among four taxa from molecular sequence data. For four taxa, there are exactly three possible unrooted binary tree topologies, making quartet inference a three-class classification problem. Maximum likelihood methods are typically used to reconstruct quartet trees, while recent studies have explored deep learning-based methods. The potential of quantum machine learning for this problem remains unexplored. In this work, we investigate quantum kernel learning for quartet phylogenetic inference from nucleotide alignments and introduce a block-wise tuned quantum kernel (BTQK) for quartet topology classification. Using training alignments from the EvoNAPS dataset, we derive 625-dimensional site-pattern frequency representations and reduce them to a 64-dimensional principal-component representation (PCA64). We employ a block-wise quantum encoding strategy in which PCA64 is partitioned into eight 8-feature blocks whose fidelity kernels are combined by equal averaging. The framework further incorporates supervised kernel tuning to better capture the underlying structure of quartet topology classes. Across ten mutually disjoint balanced training splits, BTQK achieves a Macro-F1 of 0.74 on 51,241 held-out EvoNAPS test alignments, consistently outperforming validation-tuned linear, RBF and polynomial SVMs under the same PCA64 representation. Experiments on independent TreeBASE alignments show that BTQK also outperforms the classical SVM baselines using the same representation. Although, as expected, the maximum-likelihood method remains the strongest method, these results demonstrate the potential of quantum kernel learning for quartet phylogenetic tree inference and provide a foundation for further investigation of quantum machine learning approaches in phylogenetics.

Index Terms—Quantum machine learning, Quantum kernels, Support vector machines, Phylogenetic inference, Quartet topology classification

I. INTRODUCTION

Phylogenetic inference reconstructs evolutionary relationships among taxa from molecular data [1]. Quartets, unrooted binary trees over four taxa, are the smallest informative phylogenetic units and serve as fundamental building blocks for constructing larger phylogenetic trees [2]–[4]. In this study, the four taxa correspond to four nucleotide (DNA) sequences in an alignment. For four taxa A, B, C, and D, there are exactly three possible unrooted binary tree topologies, represented by

the bipartitions (AB|CD), (AC|BD), and (AD|BC). Therefore, quartet topology inference can be formulated as a three-class classification problem.

Quartet trees are typically inferred using maximum likelihood (ML)-based methods, such as IQ-TREE 3 [5]. More recently, machine-learning approaches have also been proposed for quartet topology inference [6]. This study demonstrates that quartet tree inference can be formulated as a supervised classification problem; however, the potential of quantum-kernel representations for this task remains unexplored.

Quantum kernel methods embed classical data into quantum states and construct kernel matrices from quantum-state similarities for use with classical kernel-based learners such as support vector machines (SVMs) [7], [8]. Their performance depends on the quantum feature map, input scaling, and coefficient tuning, and multiple such kernels can be combined to capture complementary structure, as detailed in Section II.

To apply quantum kernel methods to quartet topology inference, each alignment must first be represented in a form suitable for quantum encoding. We use the relative frequencies of the 625 possible site patterns of a four-taxon alignment (Section II) for this purpose, but directly encoding all 625 features into a quantum circuit is computationally impractical. We therefore reduce the representation to 64 principal components (PCA64). The resulting features are signed and have different scales, making the quantum kernel sensitive to input scaling and circuit structure.

To address these challenges, we propose a block-wise tuned quantum kernel (BTQK) for quartet topology classification using the PCA64 representation. We compare BTQK with classical SVM kernels using the same PCA64 representation. We also investigate the effects of quantum feature maps and coefficient tuning, and evaluate split-wise stability, generalization to independent TreeBASE alignments, and computational cost.

Our contributions are threefold: (1) we introduce BTQK for quartet tree topology classification using PCA64; (2) we provide a comparison between BTQK and tuned classical SVM kernels using identical inputs, preprocessing, and training splits; and (3) we evaluate the stability and generalization of BTQK across multiple data splits and on independent empirical data. The remainder of this paper is organized as

Corresponding author: Nguyen-Viet-Dung Nghiem (email: dung.nghiem@vnu.edu.vn)

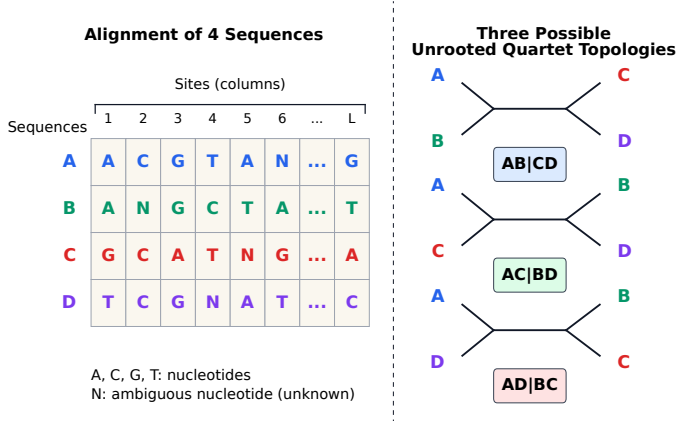


Fig. 1. Illustration of quartet topology inference from a four-sequence alignment to one of three possible unrooted quartet topologies.

follows. Section II reviews related work, data and proposed method. Section III presents the experimental setup and results. Finally, Section IV discusses the findings and limitations of the paper.

II. DATA AND METHODS

A. Data

In this study, we adopted empirical data from the IQ-NET paper [6]. Specifically, we reused 466,544 empirical quartet alignments extracted from the EvolNAPs database together with their corresponding topology labels comprising a training set of 415,303 alignments and a held-out internal test set of 51,241 alignments. We also reused an external independent test set of 116,776 quartet alignments extracted from the TreeBASE database to evaluate different methods.

B. Quartet tree inference

Given an alignment of four nucleotide sequences A, B, C, and D, quartet topology inference aims to determine which of the three possible unrooted tree topologies, (AB|CD), (AC|BD), or (AD|BC), best describes their evolutionary relationships (see Figure 1 for illustration). Maximum-likelihood methods, such as IQ-TREE 3 [5], evaluate candidate tree topologies under explicit evolutionary models and select the tree with the highest likelihood. More recently, machine-learning approaches have formulated quartet inference as a three-class classification problem and demonstrated promising results [6], [9].

C. Data Representation

Each site (column) of a quartet alignment with l sites contains one nucleotide state, A, C, G, T, or the ambiguous state N, from each of the four sequences, giving $5^4 = 625$ possible site patterns. We represent each alignment by a 625-dimensional feature vector of relative site-pattern frequencies: each feature is the number of occurrences of a pattern divided by l . This representation is invariant to site order and summarizes the site-pattern information used for classification.

D. Quantum Kernels

Quantum kernels encode classical inputs into quantum states and compute their similarities to construct kernel matrices for classical learning algorithms, such as support vector machines (SVMs) [7], [8]. This encoding is performed by a quantum feature map, which maps a classical input vector $\mathbf{x}$ to a quantum state $|\phi(\mathbf{x})\rangle$ through a parameterized quantum circuit. The resulting quantum states define a feature space in which the similarity between two inputs can be evaluated. Quantum feature maps commonly use data-dependent single-qubit rotations and two-qubit interactions, such as ZZ interactions, to encode input features. The performance of a quantum kernel depends strongly on the choice of feature map and the learning task [10]. Therefore, appropriate design and tuning are essential for achieving competitive quantum-kernel performance.

The performance of quantum kernels can also be affected by input scaling and parameter selection. Input scaling controls the effective bandwidth of a quantum kernel, determining how strongly the kernel distinguishes between different input samples [11]. Quantum feature-map parameters can also be optimized using kernel-target alignment, which measures how well the resulting kernel matrix agrees with the class labels [12], [13]. A higher kernel-target alignment score indicates better agreement between the kernel matrix and the class labels.

Multiple-kernel learning combines several kernels to improve learning performance [14]. The same idea has been extended to quantum kernels, where multiple quantum feature maps are combined into a single kernel [15]. A quantum multi-view approach trains a separate quantum circuit for each data view and learns how to combine their kernels [16]. These considerations motivate the block-wise quantum-kernel construction introduced next.

E. Block-wise tuned quantum kernel (BTQK)

Directly simulating a 64-qubit quantum circuit for the PCA64 representation is computationally impractical. To address this limitation, BTQK combines two existing ideas: block-wise quantum encoding, which partitions high-dimensional inputs into smaller quantum circuits [17], and multiple-kernel learning, which combines multiple kernels additively [14], [15]. Specifically, BTQK divides each PCA64 vector into eight blocks and constructs a quantum kernel for each block. For each block, a quantum feature map ($U(\mathbf{x})$) encodes the input features into a quantum state ($|\phi(\mathbf{x})\rangle$), and pairwise state fidelities are used to construct the kernel matrix [7], [8]. The encoding coefficients are tuned separately for each block. The resulting eight kernels are then averaged with equal weights to form a single precomputed kernel for a support vector machine (SVM) [18].

Technically, let $\mathbf{x} = (x_0, \dots, x_{63})^\top \in \mathbb{R}^{64}$ denote the PCA64 representation of a quartet. Each component is robustly standardized using training-split statistics, clipped to $[-3, 3]$,

and mapped linearly to a bounded angle

$$s_j(\mathbf{x}) = \text{clip} \left(\frac{x_j - \text{median}_{\text{tr}}(x_j)}{\text{IQR}_{\text{tr}}(x_j)}, -3, 3 \right), \theta_j = \frac{\pi}{2} \frac{s_j + 3}{6} \quad (1)$$

where $\text{median}_{\text{tr}}(x_j)$ and $\text{IQR}_{\text{tr}}(x_j)$ are the median and interquartile range of feature j , respectively, computed from the training data. This produces the angle vector $\boldsymbol{\theta} = (\theta_0, \dots, \theta_{63})^\top$, where $0 \leq \theta_i \leq \pi/2$. Thus, each block b ($b = 0, \dots, 7$) contains the eight angles $\theta_{8b}, \dots, \theta_{8b+7}$, and qubit q receives angle θ_{8b+q} . The exact circuit family is selected empirically in Section III-B. In general, a candidate block encoder may use one or more data-dependent rotation gates,

$$R_A(\lambda_{b,A} \theta_{8b+q}), \quad A \in \{X, Y, Z\}, \quad q = 0, \dots, 7, \quad (2)$$

where $\lambda_{b,A}$ is a block-level angle coefficient shared by all qubits for rotation axis A in block b . Candidates may optionally include a nearest-neighbor interaction layer after the rotations. We consider a data-independent CNOT chain and a data-dependent ZZ (DZZ) interaction [8]. For DZZ, adjacent qubits use angle $(\delta_b/\pi) \theta_{8b+q} \theta_{8b+q+1}$, with block-specific δ_b .

Let $U_b(\mathbf{x}; \boldsymbol{\eta}_b)$ denote the resulting eight-qubit feature map for block b , where $\boldsymbol{\eta}_b$ collects the available rotation and interaction coefficients of the selected candidate. This circuit maps the input block to a quantum state

$$|\phi_b(\mathbf{x}; \boldsymbol{\eta}_b)\rangle = U_b(\mathbf{x}; \boldsymbol{\eta}_b)|0\rangle^{\otimes 8}. \quad (3)$$

For two PCA64 inputs $\mathbf{x}, \mathbf{z} \in \mathbb{R}^{64}$, the block kernel is the squared inner product between their two block states:

$$k_b(\mathbf{x}, \mathbf{z}; \boldsymbol{\eta}_b) = |\langle \phi_b(\mathbf{x}; \boldsymbol{\eta}_b) | \phi_b(\mathbf{z}; \boldsymbol{\eta}_b) \rangle|^2. \quad (4)$$

Thus, each block kernel is simply a similarity score between 0 and 1, computed from the two quantum states produced by the same block circuit.

The coefficient vector $\boldsymbol{\eta} = (\boldsymbol{\eta}_0, \dots, \boldsymbol{\eta}_7)$ collects the block-level coefficients across all eight blocks. BTQK combines the eight block kernels through the equal-weight average

$$k_{\boldsymbol{\eta}}(\mathbf{x}, \mathbf{z}) = \frac{1}{8} \sum_{b=0}^7 k_b(\mathbf{x}, \mathbf{z}; \boldsymbol{\eta}_b). \quad (5)$$

Because each k_b is a positive-semidefinite kernel, their non-negative average is also a valid kernel. Unlike a block-product kernel, in which the global similarity is the product of the block fidelities [17], the additive construction prevents a single low-similarity block from suppressing the contributions of the remaining blocks. It also mitigates the contraction of off-diagonal similarities that can occur when many fidelity values smaller than one are multiplied.

The coefficients in $\boldsymbol{\eta}$ act as angle-scaling parameters for the quantum feature map. Since input scaling can strongly affect quantum-kernel bandwidth [11], BTQK selects these coefficients from the training split before fitting the final SVM. The coefficients are selected using a kernel-alignment criterion [12]: a useful training kernel should assign larger

similarities to samples from the same topology class than to samples from different classes.

For a training set $\mathcal{D}_{\text{tr}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, the Gram matrix associated with a candidate coefficient vector $\boldsymbol{\eta}$ is defined as $[\mathbf{K}_{\boldsymbol{\eta}}]_{ij} = k_{\boldsymbol{\eta}}(\mathbf{x}_i, \mathbf{x}_j)$. This desired class geometry is encoded by a label-similarity matrix $\mathbf{Y} \in \mathbb{R}^{n \times n}$

$$[\mathbf{Y}]_{ij} = w_i w_j \begin{cases} 1, & y_i = y_j, \\ -\frac{1}{L-1}, & y_i \neq y_j, \end{cases} \quad w_i = \frac{1}{\sqrt{n_{y_i}}}, \quad (6)$$

where $L = 3$ is the three topology classes and n_{y_i} is the number of training samples in the class of sample i . Same-class pairs are positive, different-class pairs are negative, and the weights keep the three classes balanced.

BTQK then searches finite coefficient grids and keeps settings that improve the alignment between the quantum kernel $\mathbf{K}_{\boldsymbol{\eta}}$ and this label matrix

$$\boldsymbol{\eta}^* \approx \arg \max_{\boldsymbol{\eta} \in \mathcal{A}} \text{Align}(\mathbf{K}_{\boldsymbol{\eta}}, \mathbf{Y}). \quad (7)$$

Here, $\mathcal{A}$ denotes the finite grids explored by coordinate search. The alignment score is centered kernel alignment [13], i.e., the normalized dot product between centered versions of $\mathbf{K}_{\boldsymbol{\eta}}$ and $\mathbf{Y}$. A higher score means that the kernel better matches the class structure. No test labels are used in this step; validation labels are used only for model selection, and the test set is reserved for final evaluation.

F. Classical Baselines

The classical baselines are designed to isolate the effect of kernel choice. BTQK and the baselines use the same downstream SVM framework, differing primarily in the kernel. To make the comparison stricter, the SVM baselines use the same train-fitted angle preprocessing applied before BTQK's circuit: robust standardization, clipping to $[-3, 3]$, and mapping to $[0, \pi/2]$; the extra SVM-pipeline scaler is disabled. We evaluate linear, RBF, and third-degree polynomial (Poly3) SVMs. For each split, all configurations in a fixed validation grid over C , γ , and `coef0` are fit on the 500 training samples; the validation-best configuration within each SVM family is then evaluated once on the test set. Table I reports the grid used for the classical baselines.

TABLE I
HYPERPARAMETER GRID FOR THE CLASSICAL SVM BASELINES

Model	Hyperparameter candidates
Linear	$C \in \{0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30, 100\}$
RBF	C : same as Linear; $\gamma \in \{\text{scale}, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0\}$
Poly3	degree = 3; C : same as Linear; $\gamma \in \{\text{scale}, 0.003, 0.01, 0.03, 0.1\}$; <code>coef0</code> $\in \{0, 1\}$

We also include a classical block-RBF control to test whether BTQK's gain is due only to preprocessing, block partitioning, and kernel averaging. This control keeps the same eight-block partition and equal-averaging rule but replaces each quantum block fidelity kernel with an RBF

kernel. The block bandwidths are selected from the training split using the same alignment procedure, with $\gamma_b \in \{\text{scale}, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0\}$, while C is selected on validation from the same grid as the global RBF SVM baseline in Table I.

III. EXPERIMENTAL EVALUATION

A. Evaluation Protocol

The PCA transformation is fitted once on the 415,303 alignments without using topology labels. The internal and external test sets are projected using the same fixed PCA transformation. We retain the PCA64 representation, which explains 99.09% of the variance in the training data. These components are divided into eight consecutive blocks of eight features, preserving the PCA variance ordering. This block-wise representation enables tractable state-vector simulation, as directly encoding all 625 site-pattern features or the full PCA64 vector in a single quantum circuit is computationally impractical.

We used 5,000 alignments for training, which were divided into ten mutually disjoint and balanced splits of 500 alignments each. Each split contained 167, 167, and 166 alignments for the three quartet topology classes, respectively. We also held out a balanced validation set of 41,529 alignments (13,843 per topology class). We limited each training split to 500 alignments because fidelity-kernel construction scales quadratically with the training size, while test evaluation scales as $O(n_{\text{train}}n_{\text{test}})$. Pilot experiments with 1,000 and 2,000 training alignments achieved higher Macro-F1 scores but substantially increased computational time, exceeding our available computational capacity.

For BTQK, model fitting and kernel-coefficient estimation use only the training split, with coefficients selected by the alignment objective in Section II-E; circuit architectures and SVM hyperparameters are selected on validation data. Classical SVMs follow the same validation-based model-selection protocol, and each selected configuration is evaluated on the held-out test set. Performance is measured by accuracy and Macro-F1, with paired Wilcoxon signed-rank tests and 95% percentile-bootstrap confidence intervals for mean paired differences across the ten splits.

Quantum kernels were evaluated by CPU state-vector simulation. Experiments use Python 3.10.12, PennyLane 0.42.3, scikit-learn 1.7.2, NumPy 2.2.6, pandas 2.3.3, and SciPy 1.15.3 on an Intel Xeon Bronze 3104 node with six CPU cores. Code and experimental configurations are publicly available at <https://github.com/dung321046/BTQK>.

B. Feature-Map Selection

This subsection selects the circuit used in each BTQK block while keeping the data representation, block partition, kernel averaging and SVM protocol fixed. We evaluate nine configurations formed by three rotation structures: RY , $RY-RZ$, and $RX-RY-RZ$, combined with three interaction choices: none, CNOT and DZZ, as defined in Section II-E. Each candidate is evaluated both without tuning, with all available coefficients

TABLE II
FEATURE-MAP SELECTION ON PCA64. VALUES ARE REPORTED AS MEAN $\pm$ STANDARD DEVIATION OVER THE 10 VALIDATION RUNS

Candidate	Without tuning Macro-F1	Tuned Macro-F1
RY-RZ/DZZ	0.7138 $\pm$ 0.0075	0.7446 $\pm$ 0.0079
RY-RZ/none	0.7137 $\pm$ 0.0070	0.7445 $\pm$ 0.0079
RY-RZ/CNOT	0.7137 $\pm$ 0.0070	0.7445 $\pm$ 0.0079
RY/none	0.7082 $\pm$ 0.0097	0.7385 $\pm$ 0.0099
RY/DZZ	0.7075 $\pm$ 0.0098	0.7385 $\pm$ 0.0099
RY/CNOT	0.7082 $\pm$ 0.0097	0.7385 $\pm$ 0.0099
RX-RY-RZ/none	0.6996 $\pm$ 0.0099	0.7124 $\pm$ 0.0106
RX-RY-RZ/CNOT	0.6996 $\pm$ 0.0099	0.7124 $\pm$ 0.0106
RX-RY-RZ/DZZ	0.7032 $\pm$ 0.0092	0.7124 $\pm$ 0.0106

fixed to one and with training-only tuning. For each block, the RX and RY rotation coefficients are searched over the same grid $\alpha_b \in \{0.5, 0.7, 0.9, 1.1\}$, the RZ coefficient is searched over $\beta_b \in \{0.175, 0.270, 0.365, 0.460, 0.555, 0.650\}$ and for DZZ $\delta_b \in \{0, 0.25, 0.50, 0.75, 1.00\}$. During circuit screening, a fixed reference value $C = 0.5$ is used for all candidates so that the comparison varies only the circuit configuration. Validation Macro-F1 is the primary selection criterion, with runtime and circuit simplicity used when performance is practically indistinguishable; the test set is not accessed.

Table II shows that the $RY-RZ$ family performs best both before and after training-only coefficient tuning. $RY-RZ/DZZ$ achieves the highest validation Macro-F1, but exceeds $RY-RZ/none$ by only 0.0001, far below the split-to-split variation. For the tuned variants, the corresponding evaluation times are approximately 25, 20, and 27 ms per sample for $RY-RZ/DZZ$, $RY-RZ/none$ and $RY-RZ/CNOT$, respectively. We thus select $RY-RZ/none$ as the final architecture because it achieves essentially identical validation performance with lower evaluation cost and a simpler circuit structure. The complete feature-map screening across all nine candidate configurations and ten training splits, including both untuned and tuned evaluations, required approximately 55.1 hours.

The identical CNOT and no-interaction results are expected rather than a duplication artifact: in this one-layer construction, CNOT is a data-independent unitary applied after the data-dependent rotations and therefore preserves fidelity when applied to both samples. Thus, CNOT increases evaluation cost without changing the resulting kernel. DZZ, by contrast, can alter the kernel through data-dependent interaction angles, but its negligible performance gain does not justify the additional circuit complexity. Although circuit structure and data encoding can affect model expressivity [19], [20], the tested two-qubit interactions provide no measurable benefit over the separable $RY-RZ$ map.

With the $RY-RZ/none$ architecture fixed, the SVM regularization parameter is then tuned on the validation set over $C \in \{0.1, 0.2, 0.3, 0.5, 0.7, 1, 2, 3\}$. This post-selection search chooses $C = 0.5$. After selecting $C = 0.5$, the training-only coefficient search is rerun independently

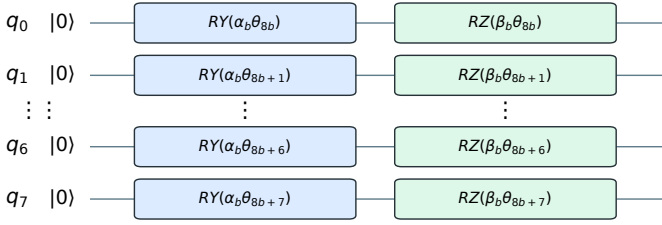


Fig. 2. Schematic of one block of the selected RY - RZ /none encoder: each qubit receives a data-encoded RY -then- RZ rotation pair (Eq. 2) with no entangling gates. Only the first two and last two of the eight qubits (q_0, q_1, q_6, q_7) are shown; the remaining four are structurally identical and omitted for compactness.

TABLE III
CLASSIFICATION PERFORMANCE ON THE INTERNAL TEST SET.
LEARNING-BASED MODELS USE PCA64

Method	Accuracy	Macro-F1
IQ-TREE HKY+F	0.8108	0.8108
BTQK	0.7427 ± 0.0070	0.7428 ± 0.0070
BTQK without tuning	0.7126 ± 0.0070	0.7127 ± 0.0070
RBF SVM	0.7074 ± 0.0085	0.7072 ± 0.0084
Third-degree polynomial SVM	0.7070 ± 0.0053	0.7069 ± 0.0053
Linear SVM	0.7048 ± 0.0060	0.7047 ± 0.0061
Classical block-RBF control	0.6938 ± 0.0109	0.6939 ± 0.0109

on each of the 10 training splits. The modal coefficients are $\alpha = (0.5, 0.5, 0.5, 1.1, 0.5, 0.5, 0.5, 0.5)$ and $\beta = (0.175, 0.175, 0.175, 0.650, 0.175, 0.175, 0.175, 0.175)$, appearing in 9/10 splits; the remaining split changes only α_1 from 0.5 to 0.7. The final BTQK therefore uses the selected RY - RZ block encoder, equal averaging of the eight block kernels, and split-wise training-only coefficient selection. Figure 2 illustrates one block of the final encoder.

C. Comparison with Classical Kernels

With the circuit family and coefficient-selection procedure fixed, we evaluated BTQK on the PCA64 internal test set against the three SVM baselines, using the post-selection validation choice $C = 0.5$ from Section III-B. For PCA64 models, results are reported as mean $\pm$ std over the 10 training splits; IQ-TREE is reported once because it does not depend on the training splits.

Table III shows that BTQK achieves the highest accuracy (0.7427) and Macro-F1 (0.7428) among the compared PCA64 kernel classifiers, compared with an accuracy of 0.7074 for the RBF SVM. Coefficient tuning improves the Macro-F1 by +0.0301. The classical block-RBF control performs worse than both BTQK and the global RBF SVM, suggesting that block-wise kernel averaging alone does not account for BTQK’s improved performance. As expected, the maximum-likelihood method IQ-TREE achieves the highest overall accuracy. When using all 625 features (Original625), the RBF SVM achieves a Macro-F1 of 0.7487, which is comparable to that of BTQK (0.7428).

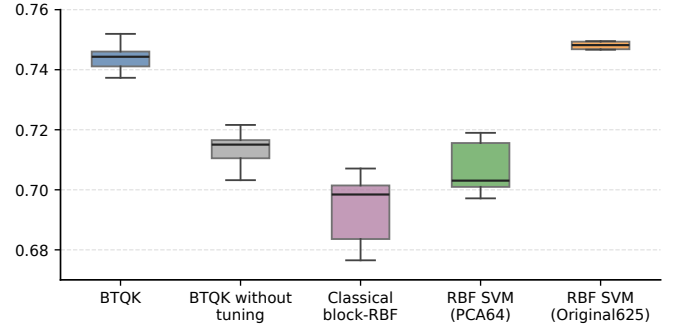


Fig. 3. Macro-F1 distributions over 10 mutually disjoint training splits for BTQK, its untuned variant, block-RBF, and RBF SVMs on PCA64 and Original625.

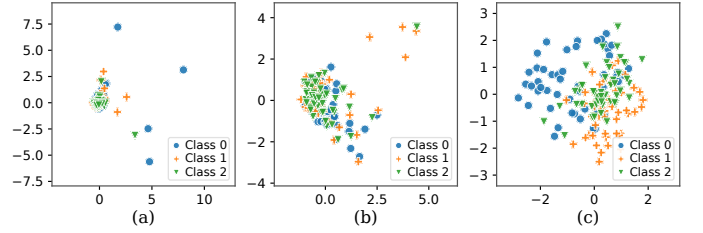


Fig. 4. Representation spaces for the same balanced subset of 150 samples. (a) raw 625-dimensional features projected to 2D using a visualization-only PCA, (b) PC1-PC2 of the fixed PCA64 representation used in the experiments, and (c) the selected BTQK kernel representation projected by kernel PCA.

Figure 3 shows the Macro-F1 distributions for BTQK, BTQK without tuning, the block-RBF control, and the RBF SVM on both PCA64 and Original625. Overall, BTQK shows a clear advantage over the classical kernel classifiers using the same PCA64 representation, while remaining comparable to the RBF SVM using Original625.

Wilcoxon signed-rank tests in Table IV show that BTQK outperforms all compared PCA64 baselines on all 10 splits, with all 95% confidence intervals above zero and two-sided $p = 0.0020$. As this is the smallest attainable p -value for $n = 10$, effect magnitudes are better compared using the mean $\Delta F1$ and confidence intervals.

TABLE IV
WILCOXON SIGNED-RANK TESTS COMPARING BTQK WITH THE BASELINES

Baseline	Mean $\Delta F1$	95% CI	p -value
Linear SVM	+0.0381	[+0.0344, +0.0418]	0.0020
Polynomial SVM	+0.0359	[+0.0326, +0.0390]	0.0020
RBF SVM	+0.0356	[+0.0308, +0.0402]	0.0020
BTQK without tuning	+0.0301	[+0.0273, +0.0329]	0.0020
Classical block-RBF control	+0.0489	[+0.0435, +0.0547]	0.0020

Figure 4 provides a qualitative comparison of the raw, PCA64, and BTQK-induced representations. The BTQK projection appears to show a more structured class geometry, although substantial overlap remains among the three topology classes.

Table V shows the performance of the different methods

TABLE V
EXTERNAL EVALUATION ON THE TREEBASE DATA

Method	Accuracy	Macro-F1
IQ-TREE HKY+F	0.8244	0.8082
BTQK	0.6520 ± 0.0177	0.6276 ± 0.0094
RBF SVM	0.6213 ± 0.0228	0.6003 ± 0.0173
Linear SVM	0.6176 ± 0.0159	0.5964 ± 0.0118
Third-degree polynomial SVM	0.6154 ± 0.0154	0.5956 ± 0.0103

on the external TreeBASE test set. BTQK remains the best-performing PCA64-based model. The accuracy of IQ-TREE remains similar to that on the internal test set, whereas the performance of all PCA64-based models decreases on TreeBASE, indicating a generalization gap between the internal and external datasets.

IV. DISCUSSION AND CONCLUSION

BTQK consistently outperforms the validation-tuned linear, RBF, and polynomial SVMs using the same PCA64 representation. The selected block-wise RY-RZ feature map, together with training-only coefficient tuning, provides an effective quantum-kernel representation for quartet topology classification. While the current study focuses on a selected set of feature maps and coefficient values, further exploration may lead to additional improvements. The selected encoder is separable and can be efficiently simulated classically. These results characterize BTQK as a quantum-kernel representation approach evaluated under classical simulation.

For the selected RY-RZ/none architecture, the total training time with coefficient tuning is approximately 1,360 seconds across all ten training splits. Once trained, BTQK requires only about 21 ms per test sample for inference. The current experiments are limited to 500 training samples per split due to our available computational capacity. Future work could improve the training pipeline to support larger training sets.

Although BTQK has not yet achieved the accuracy of maximum likelihood method and shows limited generalization to the external test set, this work establishes a foundation for quantum machine learning in quartet phylogenetic inference and opens a new research direction at the intersection of quantum computing and phylogenetic analysis. Future work will focus on improving the representation and scalability of BTQK through alternative feature representations, optimized block partitioning and kernel weighting, and scalable kernel approximations, as well as extending the framework toward implementation on realistic quantum hardware under noisy conditions.

V. ACKNOWLEDGMENTS

This work is financially supported by Vietnam National Foundation for Science and Technology Development (102.05-2025.65). AI-assisted writing tools were used to improve spelling, grammar, and language during the preparation of this manuscript.

REFERENCES

- [1] J. Felsenstein, *Inferring Phylogenies*. Sunderland, Massachusetts, USA: Sinauer Associates, 2004.
- [2] H. A. Schmidt, K. Strimmer, M. Vingron, and A. von Haeseler, "TREE-PUZZLE: Maximum likelihood phylogenetic analysis using quartets and parallel computing," *Bioinformatics*, vol. 18, no. 3, pp. 502–504, 2002.
- [3] V. Ranwez and O. Gascuel, "Quartet-based phylogenetic inference: Improvements and limits," *Molecular Biology and Evolution*, vol. 18, no. 6, pp. 1103–1116, 2001.
- [4] L. S. Vinh and A. von Haeseler, "IQPNNI: Moving fast through tree space and stopping in time," *Molecular Biology and Evolution*, vol. 21, no. 8, pp. 1565–1571, 2004.
- [5] T. K. F. Wong, N. Ly-Trong, H. Ren, P. Demotte, H. Baños, A. J. Roger, E. Susko, C. Bielow, N. De Maio, N. Goldman, M. W. Hahn, M. dos Reis, L. S. Vinh, G. Huttley, R. Lanfear, and B. Q. Minh, "IQ-TREE 3: Phylogenomic inference software using complex evolutionary models," *Molecular Biology and Evolution*, vol. 43, no. 5, p. msag117, 2026.
- [6] C. Yang, Z. Zhuang, P. Demotte, C. C. Dang, L. S. Vinh, B. Q. Minh, and N. Ly-Trong, "IQ-NET: Fast and accurate quartet phylogenetic inference using deep learning trained on empirical DNA alignments," *EcoEvoRxiv preprint*, 2025.
- [7] M. Schuld and N. Killoran, "Quantum machine learning in feature Hilbert spaces," *Physical Review Letters*, vol. 122, no. 4, p. 040504, 2019.
- [8] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, 2019.
- [9] Z. Zou, H. Zhang, Y. Guan, and J. Zhang, "Deep residual neural networks resolve quartet molecular phylogenies," *Molecular Biology and Evolution*, vol. 37, no. 5, pp. 1495–1507, 2020.
- [10] H.-Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, and J. R. McClean, "Power of data in quantum machine learning," *Nature Communications*, vol. 12, no. 1, p. 2631, 2021.
- [11] R. Shaydulin and S. M. Wild, "Importance of kernel bandwidth in quantum machine learning," *Physical Review A*, vol. 106, no. 4, p. 042407, 2022.
- [12] T. Hubregtsen, D. Wierichs, E. Gil-Fuster, P.-J. H. S. Derks, P. K. Faehrmann, and J. J. Meyer, "Training quantum embedding kernels on near-term quantum computers," *Physical Review A*, vol. 106, no. 4, p. 042431, 2022.
- [13] C. Cortes, M. Mohri, and A. Rostamizadeh, "Algorithms for learning kernels based on centered alignment," *Journal of Machine Learning Research*, vol. 13, pp. 795–828, 2012.
- [14] M. Gönen and E. Alpaydm, "Multiple kernel learning algorithms," *Journal of Machine Learning Research*, vol. 12, no. 64, pp. 2211–2268, 2011.
- [15] S. Miyabe, B. Quanz, N. Shimada, A. Mitra, T. Yamamoto *et al.*, "Quantum multiple kernel learning in financial classification tasks," *arXiv preprint arXiv:2312.00260*, 2023.
- [16] J. Li, Y. Song, S. Qin, and F. Gao, "Quantum multiview kernel learning with local information," *Physical Review Applied*, vol. 25, no. 3, p. 034084, 2026.
- [17] T. Suzuki, T. Miyazaki, T. Inarita, and T. Otsuka, "Quantum AI simulator using a hybrid CPU-FPGA approach," *Scientific Reports*, vol. 13, no. 1, p. 7735, 2023.
- [18] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [19] S. Sim, P. D. Johnson, and A. Aspuru-Guzik, "Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms," *Advanced Quantum Technologies*, vol. 2, no. 12, p. 1900070, 2019.
- [20] M. Schuld, R. Sweke, and J. J. Meyer, "The effect of data encoding on the expressive power of variational quantum-machine-learning models," *Physical Review A*, vol. 103, no. 3, p. 032430, Mar. 2021.